Open in Colab

After clicking the "Open in Colab" link, copy the notebook to your own Google Drive before getting started, or it will not save your work

One of the most straightforward tasks we can perform on a data set without labels is to find groups of data in our dataset which are similar to one another -- what we call clusters.

K-Means is one of the most popular "clustering" algorithms. K-means stores k centroids that it uses to define clusters. A point is considered to be in a particular cluster if it is closer to that cluster's centroid than any other centroid.

K-Means finds the best centroids by alternating between (1) assigning data points to clusters based on the current centroids (2) chosing centroids (points which are the center of a cluster) based on the current assignment of data points to clusters.

Dataset: MNIST

MNIST Dataset: A Brief Overview

Introduction

The **MNIST (Modified National Institute of Standards and Technology) dataset** is one of the most widely used datasets in machine learning, particularly for benchmarking image classification algorithms.

Description

- **Type**: Handwritten digit dataset
- **Size**: 70,000 grayscale images (28x28 pixels)
  - **Training set**: 60,000 images
  - **Test set**: 10,000 images
- **Classes**: 10 (Digits 0-9)
- **Format**: Each image is a 28×28 matrix with pixel values ranging from 0 (black) to 255 (white).

Why MNIST?

- Simple yet challenging enough for ML research.
- Well-structured and preprocessed, eliminating the need for extensive data cleaning.

Accessing MNIST

- Available in popular ML libraries:
  - **TensorFlow/Keras**: `tf.keras.datasets.mnist`
  - **PyTorch**: `torchvision.datasets.MNIST`
  - **Scikit-learn**: `fetch_openml('mnist_784')`

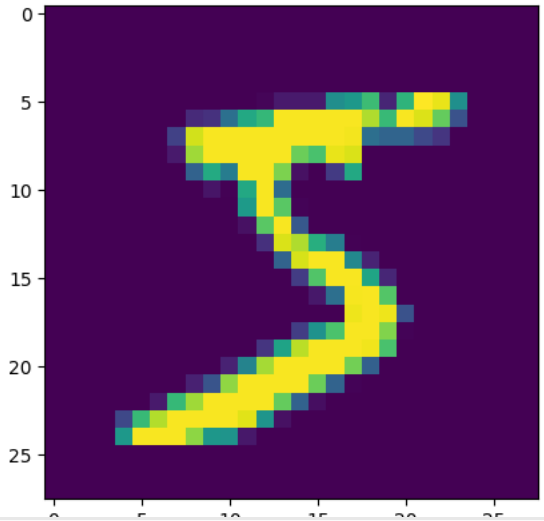
```
from sklearn.datasets import fetch_openml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.metrics import adjusted_rand_score
from sklearn.manifold import TSNE
import seaborn as sns
```

```
# Load MNIST dataset
mnist = fetch_openml('mnist_784', version = 1, parser = 'auto')
X, y = mnist['data'], mnist['target']

np.random.seed(42)
```

The MNIST digits in this set are flattened arrays of 784 pixels. We can reshape them to 28x28 pixels and plot them using matplotlib.

```
image = X.iloc[0, :].values.reshape((28, 28))
plt.imshow(image)
```

```
<matplotlib.image.AxesImage at 0x7a5170c4ca90>
```



1. Preparing the Data

a. Using the full dataset, *normalize* each column so that the minimum column value is 0, and the maximum is 1. (Hint: if your normalization process leads to missing values, replace these with 0!)

b. Get a random sample of 10% of the data. The full dataset may take a while to run some of the below methods.

```
# a.
mnist= pd.DataFrame(mnist['data'], columns=mnist['feature_names'])
```

```
for col in mnist.columns:
    min_val = mnist[col].min()
    max_val = mnist[col].max()
    mnist[col] = (mnist[col] - min_val) / (max_val - min_val)
    # Fill any NaN values with 0
    mnist[col] = mnist[col].fillna(0)
```

```
mnist.describe()
```

	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	pixel10	...	pixel775	pixel776	pixel77
count	70000.0	70000.0	70000.0	70000.0	70000.0	70000.0	70000.0	70000.0	70000.0	70000.0	...	70000.000000	70000.000000	70000.00000
mean	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000777	0.000392	0.00018
std	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.023587	0.016757	0.01100
min	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	0.000000	0.00000
25%	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	0.000000	0.00000
50%	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	0.000000	0.00000
75%	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.000000	0.000000	0.00000
max	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.000000	1.000000	1.00000

8 rows × 784 columns

```
# b.
df_sample = mnist.sample(frac=0.1, random_state=42)
X, y = df_sample.values, df_sample.index
```

2. K-Means Clustering

a. Using the MNIST subset, determine the optimal k value for k-means according to the silhouette score. Use a range of k-values from 2 - 12.

b. Fit a k-means model with the optimal k value.

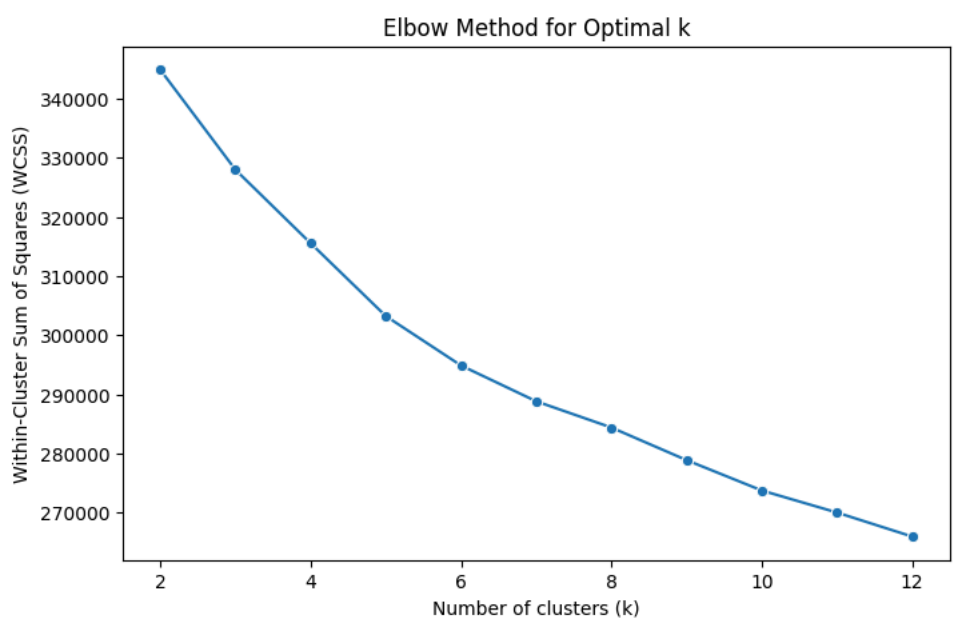
c. Using a dimensionality reduction method, t-SNE, generate a two-dimensional representation (called an **embedding**)of the MNIST subset. (See an example here. Use default values, but setting random_state = 42.)

d. Create two side-by-side scatterplots using the t-SNE represenation using plt.subplots. Color the first fig according to the true labels and the second according to k-means cluster labels. Be sure to include proper figure titles and a legend.

e. Describe the fit. Does this align with your expectation, given the silhouette score?

f. Fit k-means with 10 clusters and calculate ten slihouette score. How does the slihouette score compare with the optimal k? (If 10 is optimal, just say so.)

```
# a.
# List to store WCSS values for each k
wcss = []

# Run k-means with k values from 1 to 10 and calculate WCSS
for k in range(2, 13):
    kmeans = KMeans(n_clusters = k, n_init = 'auto', random_state = 42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)


plt.figure(figsize=(8, 5))
sns.lineplot(x = range(2, 13), y = wcss, marker = 'o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.show()
```

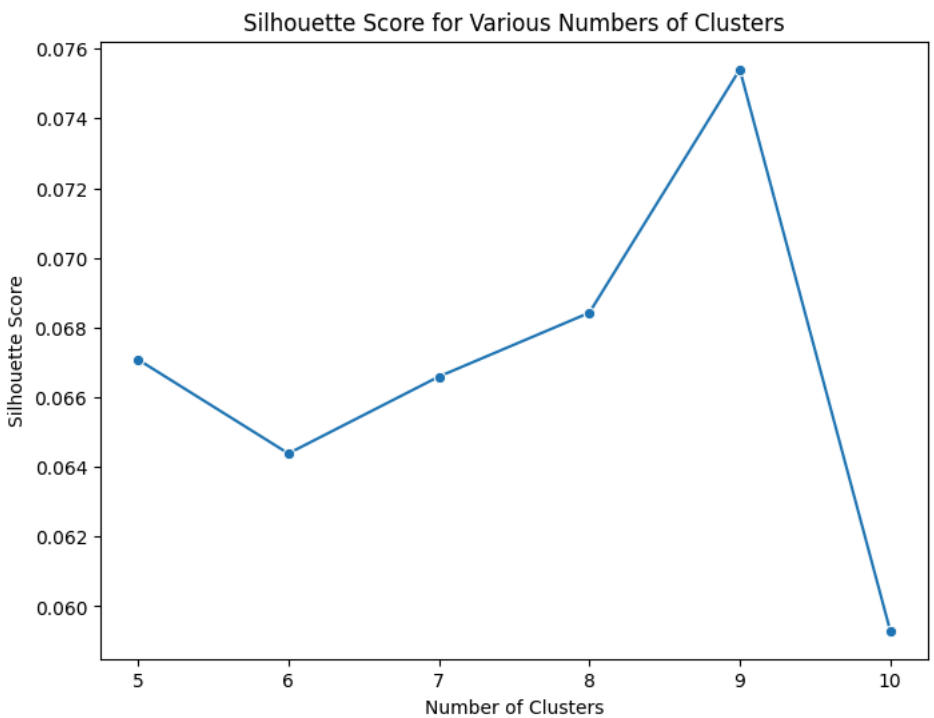


b

```
# List to store silhouette scores
silhouette_scores = []

# Range of clusters to test
range_k = range(5, 11)

# Compute silhouette scores for each number of clusters
for k in range_k:
    kmeans = KMeans(n_clusters = k, n_init = 'auto', random_state = 42)
    cluster_labels = kmeans.fit_predict(X)
    silhouette_avg = silhouette_score(X, cluster_labels)
    silhouette_scores.append(silhouette_avg)
    print(f"For k = {k}, the average silhouette score is: {silhouette_avg:.3f}")
```

```
For k = 5, the average silhouette score is: 0.067
For k = 6, the average silhouette score is: 0.064
For k = 7, the average silhouette score is: 0.067
For k = 8, the average silhouette score is: 0.068
For k = 9, the average silhouette score is: 0.075
For k = 10, the average silhouette score is: 0.059
```

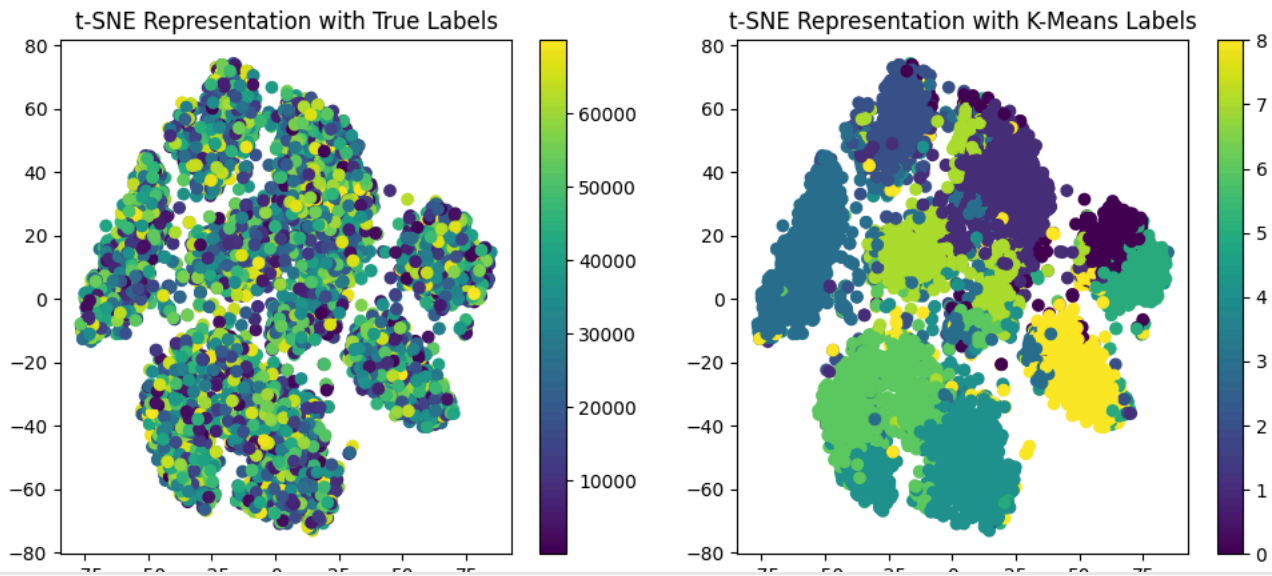
```
plt.figure(figsize=(8, 6))
sns.lineplot(x = range_k, y = silhouette_scores, marker = 'o')
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for Various Numbers of Clusters")
plt.show()
```




```
k = 9
kmeans = KMeans(n_clusters = k, random_state = 42)
kmeans.fit(X)
cluster_labels = kmeans.predict(X)
```

```
# c.
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(df_sample)
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

true_labels = y[df_sample.index.values % len(y)].astype(int)

scatter1 = ax[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_labels, cmap='viridis')
ax[0].set_title('t-SNE Representation with True Labels')

scatter2 = ax[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=cluster_labels, cmap='viridis')
ax[1].set_title('t-SNE Representation with K-Means Labels')

plt.colorbar(scatter1, ax=ax[0])
plt.colorbar(scatter2, ax=ax[1])
plt.show()
```



This looks like an ok fit in some instances but in general not very good, I wasnt really expecting this with 9 K-Mean clusters

```
kmeans_10 = KMeans(n_clusters=10, random_state=42, n_init='auto')
kmeans_10.fit(X)
cluster_labels_10 = kmeans_10.predict(X)

silhouette_avg_10 = silhouette_score(X, cluster_labels_10)

print(f"For k = 10, the average silhouette score is: {silhouette_avg_10:.3f}")
```

```
For k = 10, the average silhouette score is: 0.059
```

when looking at the average silhoutte score for optimal k = 7 and k = 10, the average is smaller than the average for k=7 meaning that k=7 provides a better clustering structure.

3. Hierarchical Clustering

Here you will be performing hierarchical clustering on the same data subset.

Here you will be comparing linkage methods and metrics. Use the following linkage/metric combinations:

Linkage Method	Metrics
Ward	Euclidean
Single	Euclidean, Cosine, Manhattan
Complete	Euclidean, Cosine, Manhattan
Average	Euclidean, Cosine, Manhattan

a. Fit hierarhiccal clustering with 10 clusters for each combination and store the silhouette scores.

b. According to the silhouette scores, which combination is optimal? Fit an HC model with this combination and store the cluster labels (make these a different variable name from that used for your k-means labels; we will eventually be comparing all clustering models).

c. As in part 2. d., plot side-by-side scatterplots using the same t-SNE embedding.

d. Describe the fit. Does this align with your expectation, given the silhouette score?

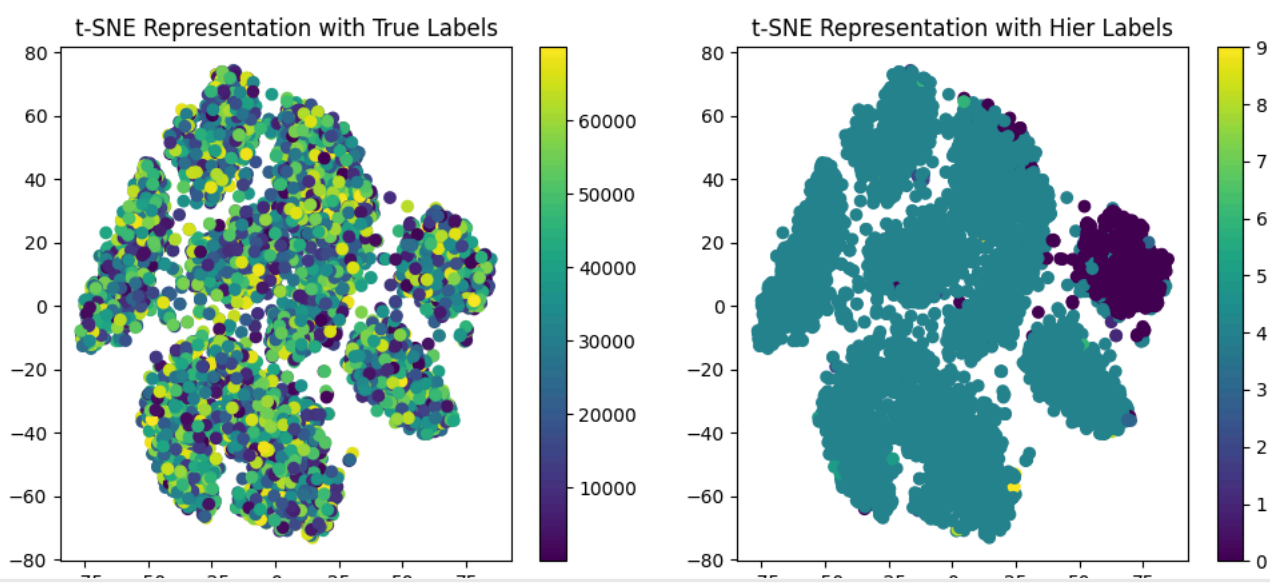
```
silhouette_scores_hc = {}

for linkage in ['ward', 'single', 'complete', 'average']:
  for metric in ['euclidean', 'cosine', 'manhattan']:
    if linkage == 'ward' and metric != 'euclidean':
      continue  # Ward only works with Euclidean

    try:
```

```
    hc = AgglomerativeClustering(n_clusters=10, metric=metric, linkage=linkage)
    cluster_labels_hc = hc.fit_predict(df_sample)
    silhouette_avg_hc = silhouette_score(df_sample, cluster_labels_hc)
    silhouette_scores_hc[(linkage, metric)] = silhouette_avg_hc
    print(f"For linkage {linkage}, metric {metric}, the average silhouette score is: {silhouette_avg_hc:.3f}")
  except Exception as e:
    print(f"Error for linkage {linkage}, metric {metric}: {e}")
```

```
For linkage ward, metric euclidean, the average silhouette score is: 0.044
For linkage single, metric euclidean, the average silhouette score is: 0.034
For linkage single, metric cosine, the average silhouette score is: -0.137
For linkage single, metric manhattan, the average silhouette score is: 0.030
For linkage complete, metric euclidean, the average silhouette score is: 0.032
For linkage complete, metric cosine, the average silhouette score is: 0.001
For linkage complete, metric manhattan, the average silhouette score is: 0.030
For linkage average, metric euclidean, the average silhouette score is: 0.071
For linkage average, metric cosine, the average silhouette score is: -0.103
For linkage average, metric manhattan, the average silhouette score is: 0.070
```

```
# b. The best combination is linkage average using the euclidean metric
hc = AgglomerativeClustering(n_clusters = 10, metric = 'euclidean', linkage = 'average')
y_clusters = hc.fit_predict(X)
```

```
# c.
tsne = TSNE(n_components=2, random_state=42)
df_sample_np = df_sample.to_numpy()
X_tsne = tsne.fit_transform(df_sample_np)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

scatter1 = ax[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_labels, cmap='viridis')
ax[0].set_title('t-SNE Representation with True Labels')

scatter2 = ax[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_clusters, cmap='viridis')
ax[1].set_title('t-SNE Representation with Hier Labels')

plt.colorbar(scatter1, ax=ax[0])
plt.colorbar(scatter2, ax=ax[1])
plt.show()
```



Start coding or generate with AI.

D: To me this seems to be very over fitting and not very useful as most of the data points are under one label with very few if any under other labels

4. DBSCAN Clustering

DBSCAN can be a little tricky to tune due to its two hyperparameters, `eps` and `min_samples`. The `eps` parameter is the maximum distance between two samples for one to be considered as in the neighborhood of the other. The `min_samples` parameter is the number of samples in a neighborhood for a point to be considered as a core point. Here, we will attempt to find optimal hyperparameter combinations, focusing primarily on `eps`.

a. Setting `min_samples` A rule of thumb is to set `min_samples` to be the number of dimensions (columns) in the dataset plus one. Here, you will try two different values:

1. Follow the rule of thumb.
2. Set `min_samples` to the number of observations in the sample divided by 10 (roughly equal numbers per digit label).

For each `min_samples` value **k** (both cases):

- i. Calculate the distance between every data point and its **kth closest neighbor**.
- ii. Compute the average **kth nearest neighbor** distances (for both `min_samples` values).
- iii. Compute the standard deviation of the **kth nearest neighbor** distances (for both `min_samples` values).
- iv. Generate a set of 10 `eps` values equally spaced between **mean ± standard deviation** (for each `min_samples` value).

b. Using the sets of `min_samples` and `eps` values, apply DBSCAN and record all silhouette scores.

c. Fit DBSCAN using the best hyperparameters based on the silhouette scores.

d. Plot the t-SNE embedded values using subplots:

- One colored by **true labels**.
- Another colored by **cluster labels**.

e. Describe the clustering results and comment on your findings.

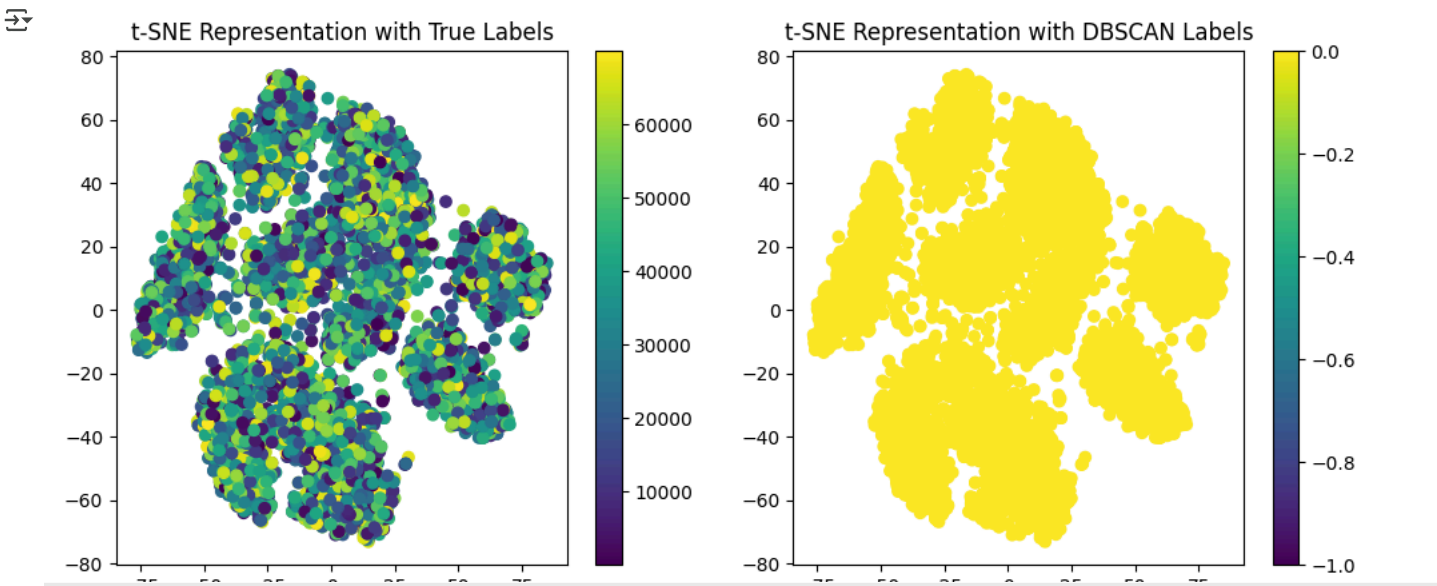
```
from sklearn.neighbors import NearestNeighbors


# a.


df_sample_np = df_sample.to_numpy()
n_dims = df_sample.shape[1]
min_samples_rule = n_dims + 1
min_samples_obs = df_sample.shape[0] // 10

min_samples_values = [min_samples_rule, min_samples_obs]

for min_samples in min_samples_values:
    nbrs = NearestNeighbors(n_neighbors=min_samples).fit(df_sample_np)
    distances, indices = nbrs.kneighbors(df_sample_np)
    kth_nearest_neighbor_distances = distances[:, -1]
    avg_kth_distance = np.mean(kth_nearest_neighbor_distances)
    std_kth_distance = np.std(kth_nearest_neighbor_distances)

    eps_values = np.linspace(avg_kth_distance - std_kth_distance, avg_kth_distance + std_kth_distance, 10)



# b.
silhouette_scores_dbscan = []
best_silhouette = -1  # Initialize best_silhouette to a very low value
best_eps = None
best_min_samples = None

for min_samples in min_samples_values:
    nbrs = NearestNeighbors(n_neighbors=min_samples).fit(df_sample_np)
    distances, indices = nbrs.kneighbors(df_sample_np)
    kth_nearest_neighbor_distances = distances[:, -1]
    avg_kth_distance = np.mean(kth_nearest_neighbor_distances)
    std_kth_distance = np.std(kth_nearest_neighbor_distances)
    eps_values = np.linspace(avg_kth_distance - std_kth_distance, avg_kth_distance + std_kth_distance, 10)

    for eps in eps_values:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        cluster_labels = dbscan.fit_predict(df_sample_np)

        # Check if there are at least 2 clusters to calculate silhouette score
        if len(np.unique(cluster_labels)) > 1:
            silhouette_avg = silhouette_score(df_sample_np, cluster_labels)

            if silhouette_avg > best_silhouette:
```

```
                best_silhouette = silhouette_avg
                best_eps = eps
                best_min_samples = min_samples

print(f"Best eps: {best_eps}, Best min_samples: {best_min_samples}, Best silhouette score: {best_silhouette}")
```

```
Best eps: 9.778306107989666, Best min_samples: 785, Best silhouette score: 0.24846880147708453
```

```
# c.
dbscan = DBSCAN(eps=best_eps, min_samples=best_min_samples)
cluster_labels_dbscan = dbscan.fit_predict(df_sample_np)
```

```
# d.
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(df_sample_np)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
scatter1 = ax[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_labels, cmap='viridis')
ax[0].set_title('t-SNE Representation with True Labels')

scatter2 = ax[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=cluster_labels_dbscan, cmap='viridis')
ax[1].set_title('t-SNE Representation with DBSCAN Labels')

plt.colorbar(scatter1, ax=ax[0])
plt.colorbar(scatter2, ax=ax[1])
plt.show()
```



Part e comments here: By far this is the worst model as all the labels are the same

5. Adjusted Rand Index (ARI) in Clustering

The **Adjusted Rand Index (ARI)** is a metric used to evaluate the similarity between two clusterings, accounting for chance. It measures how well a clustering algorithm's results match a known ground truth or reference clustering.

Intuition

- Suppose you have a set of data points, and you classify them into groups (clusters).
- ARI compares your clustering to a "correct" classification and checks how often pairs of points are **correctly grouped together** or **correctly separated** in both cases.
- A simple **Rand Index** gives a raw similarity score, but ARI adjusts for randomness—ensuring that random assignments don't get an artificially high score.

How It's Used

- **Benchmarking Clustering Algorithms:** ARI is useful for comparing different clustering methods to a gold-standard classification.

- **No Bias Toward Number of Clusters:** Unlike some other metrics, ARI corrects for the number of clusters, making it more reliable when clusters differ in size or number.

Interpreting Results

- **Range:** ARI ranges from **-1 to 1**.
  - **1** = Perfect agreement with ground truth
  - **0** = Random clustering
  - **Negative values** = Worse than random (unlikely in practice)
- **Application:** Used in cases like image segmentation, document clustering, or biological data grouping, where a reference classification exists.

Python Implementation

- **Import** via sklearn.metrics (the function `adjusted_rand_score`)
- **Inputs**: True Labels, Predicted Labels (E.g., cluster labels)
- **Output**: The Adjusted Rand Index Value

Questions:

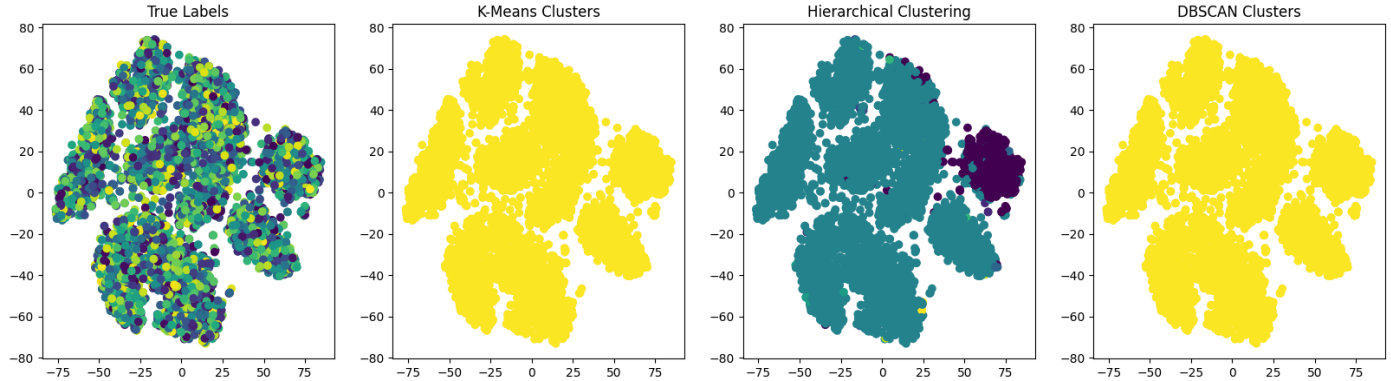
a. For each clustering method (k-means, hierarchical clustering, DBSCAN), calculate the adjusted rand index using the models with optimal hyperparameters.

b. Plot all four scatterplots using subplots (1 row, 4 columns). The first colored by the true values, and the rest colored by your cluster labels (k-means, HC, DBSCAN).

c. Do the scores seem reflecive of the assigned cluster labels, according to the plots? Please explain.

```
# a.



# Calculate Adjusted Rand Index for K-Means
ari_kmeans = adjusted_rand_score(true_labels, cluster_labels)
print(f"Adjusted Rand Index for K-Means: {ari_kmeans:.3f}")

# Calculate Adjusted Rand Index for Hierarchical Clustering
ari_hc = adjusted_rand_score(true_labels, y_clusters)
print(f"Adjusted Rand Index for Hierarchical Clustering: {ari_hc:.3f}")

# Calculate Adjusted Rand Index for DBSCAN
ari_dbscan = adjusted_rand_score(true_labels, cluster_labels_dbscan)
print(f"Adjusted Rand Index for DBSCAN: {ari_dbscan:.3f}")

# b. Plot all four scatterplots
fig, ax = plt.subplots(1, 4, figsize=(20, 5))

# True labels
ax[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=true_labels, cmap='viridis')
ax[0].set_title('True Labels')

# K-Means
ax[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=cluster_labels, cmap='viridis')
ax[1].set_title('K-Means Clusters')

# Hierarchical Clustering
ax[2].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_clusters, cmap='viridis')
ax[2].set_title('Hierarchical Clustering')

# DBSCAN
ax[3].scatter(X_tsne[:, 0], X_tsne[:, 1], c=cluster_labels_dbscan, cmap='viridis')
ax[3].set_title('DBSCAN Clusters')

plt.show()
```

```
Adjusted Rand Index for K-Means: 0.000
Adjusted Rand Index for Hierarchical Clustering: 0.000
Adjusted Rand Index for DBSCAN: 0.000
```




```
# b.
```

Part c comments here:

from the interpretation facts given in the describtion above, it appears as though this is just random clustering.

6. Visualizing Clustered Digits

Pick one of the cluster labels from one of the clustering methods (your choice). Select a random subset of 20 points belonging to that cluster.

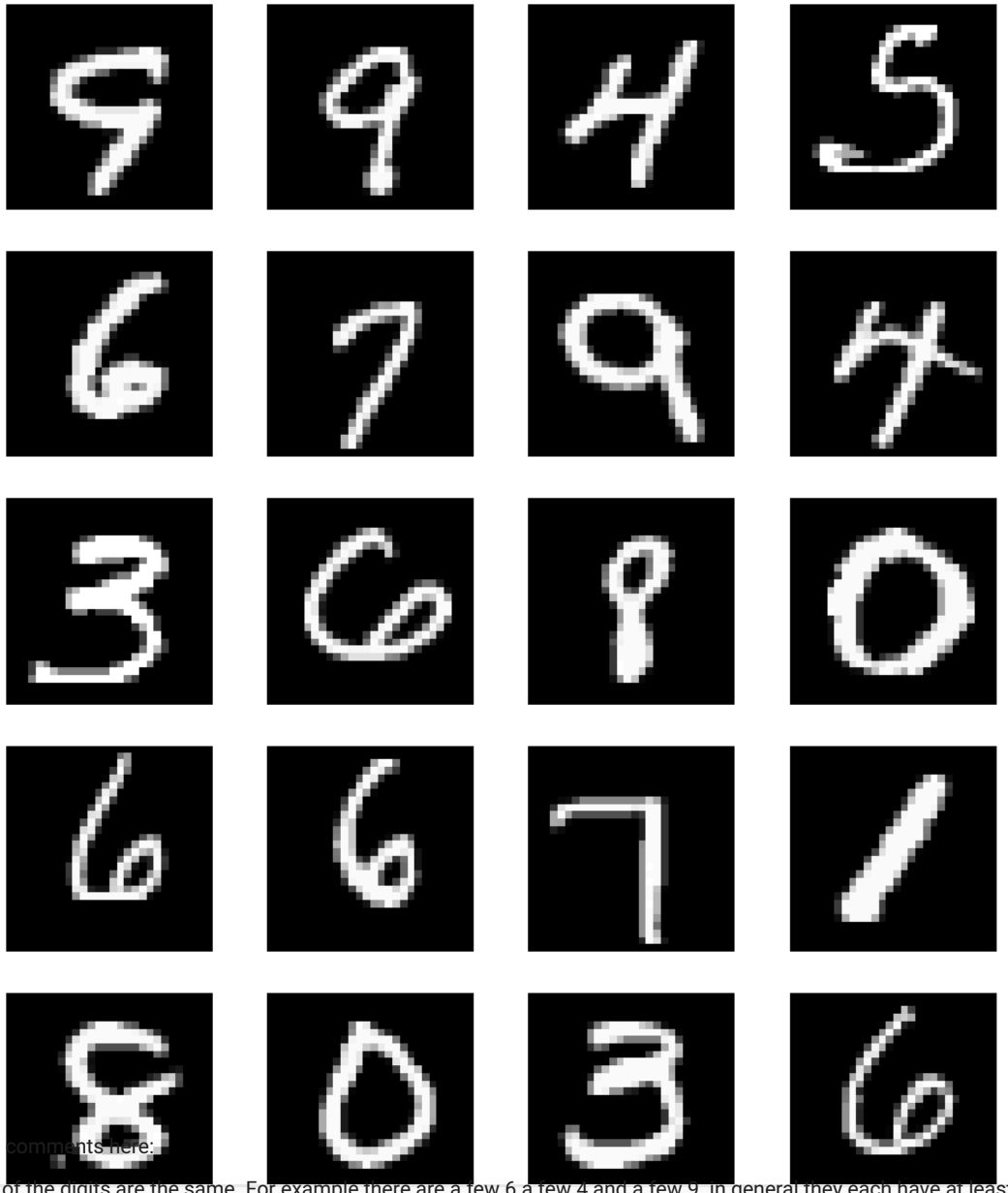
a. Using subplots (5 rows, 4 columns), plot the 20 digit images.
b. Comment on the clustered points (e.g., do they look similar?, are the representing the same digit?, etc.)

```
# a.
```

Generate | Pick one of the cluster labels from one of the clustering methods (your choice). Select a random subset of 20 points belonging to that cluster. Using subplots (5 rows, 4 columns), plot the 20 digit images. | Close

1 of 1 | Undo Changes | Use code with caution

```
chosen_cluster = 0

# Find indices of points belonging to the chosen cluster
cluster_indices = np.where(cluster_labels == chosen_cluster)[0]

# Select a random subset of 20 points
if len(cluster_indices) >= 20:
    selected_indices = np.random.choice(cluster_indices, size=20, replace=False)
else:
    selected_indices = cluster_indices

# Create subplots
fig, axes = plt.subplots(5, 4, figsize=(10, 12))

# Plot the digit images
for i, index in enumerate(selected_indices):
    image = X[index].reshape((28, 28))
    row = i // 4
    col = i % 4
    axes[row, col].imshow(image, cmap='gray')
    axes[row, col].axis('off')

plt.show()
```



Part b comments here:

some of the digits are the same. For example there are a few 6 a few 4 and a few 9. In general they each have at least something in common